

EKS Cluster Connectivity Troubleshooting SOP

Objective

This document explains how connectivity issues between an EC2 administration machine and an Amazon EKS cluster were diagnosed and resolved.

Environment

AWS Region

ap-south-1 (Mumbai)

EKS Cluster

Cluster Name: mani-eks-cluster

Version: 1.35

Endpoint Access: Public and Private

EC2 Administration Machine

Purpose: EKS Administration Workstation

IAM Role: log-role

Security Group: launch-wizard-1

Initial Problem

After successfully creating the EKS cluster, kubectl commands were executed from the EC2 administration machine.

Example:

```
kubectl get nodes
```

The command did not return any result and eventually failed.

Error:

```
dial tcp 172.31.36.226:443: i/o timeout
```

Troubleshooting Process

Step 1: Verify AWS Credentials

Verify the EC2 instance IAM role.

```
aws sts get-caller-identity
```

Output:

```
{  
  "Arn": "arn:aws:sts::447633182123:assumed-role/log-role/..."  
}
```

Result:

AWS credentials working successfully.

Conclusion:

IAM Role attached correctly.

AWS CLI functioning properly.

Step 2: Verify EKS Cluster Status

Check cluster status.

```
aws eks describe-cluster \  
--name mani-eks-cluster \  
--region ap-south-1
```

Result:

Cluster Status = ACTIVE

Conclusion:

EKS cluster created successfully.

Step 3: Verify kubectl Configuration

Check current context.

```
kubectl config current-context
```

Output:

```
arn:aws:eks:ap-south-1:447633182123:cluster/mani-eks-cluster
```

Conclusion:

```
kubectl configured correctly.
```

Step 4: Verify VPC and Subnet Placement

Check EC2 networking.

```
aws ec2 describe-instances
```

Result:

VPC:

```
vpc-011568294a640b9ed
```

Subnet:

```
subnet-03b34759d8b45a74c
```

Check EKS networking.

```
aws eks describe-cluster
```

Result:

Same VPC

Same subnet family

Conclusion:

Not a VPC routing issue.

Step 5: Verify DNS Resolution

Resolve EKS endpoint.

```
nslookup <eks-endpoint>
```

Output:

```
172.31.11.165
```

```
172.31.36.226
```

Observation:

EKS endpoint resolving to private control plane ENIs.

Conclusion:

DNS functioning correctly.

Step 6: Verify Network Connectivity

Test connection to EKS control plane.

```
Test-NetConnection 172.31.36.226 -Port 443
```

Result:

```
TcpTestSucceeded : False
```

Conclusion:

Traffic blocked before reaching EKS API.

Root Cause #1

Security Group Restriction

EKS automatically created:

```
sg-0c023fc21e49937a6
```

Cluster Security Group contained only:

Source:

sg-0c023fc21e49937a6

Meaning:

Cluster trusted itself only.

Administration EC2 used:

sg-00fdde69888e28ccd

launch-wizard-1

Traffic Flow:

EC2



launch-wizard-1 SG



EKS Cluster SG



Denied

Fix #1

Add inbound rule to Cluster Security Group.

Navigate:

VPC

→ Security Groups

→ EKS Cluster Security Group

→ Inbound Rules

Add:

Type:

HTTPS

Protocol:

TCP

Port:

443

Source:

sg-00fdde69888e28ccd
(launch-wizard-1)

Result:

EC2 successfully reached EKS API endpoint.

New Error:

You must be logged in to the server

Root Cause #2

EKS Authorization Failure

Network connectivity was now working.

However EKS authorization failed.

Verify identity:

```
aws sts get-caller-identity
```

Result:

Role:

log-role

Check EKS Access Entries.

Configured:

root

AWSServiceRoleForAmazonEKS

Missing:

log-role

EKS received valid authentication token but did not authorize access.

Authentication vs Authorization

Authentication:

Who are you?

Authorization:

What are you allowed to do?

In this case:

Authentication = SUCCESS

Authorization = FAILURE

Fix #2

Create EKS Access Entry.

Navigate:

EKS

→ Cluster

→ Access

→ Create Access Entry

Principal:

arn:aws:iam::447633182123:role/log-role

Type:

Standard

Attach Policy:

AmazonEKSClusterAdminPolicy

Scope:

Cluster

Save.

Validation

Run:

```
kubectl get nodes
```

Expected:

No resources found

or

Node list displayed

depending on node group status.

Run:

```
kubectl get pods -A
```

Expected:

kube-system pods visible

Result:

Cluster access restored successfully.

Key Learning

EKS access requires three independent layers.

Layer 1: Network Access

EC2

→

EKS API Endpoint

Controlled by:

Security Groups

NACLs

Routing

Layer 2: Authentication

IAM Role

→

EKS Token

Controlled by:

AWS IAM

STS

Layer 3: Authorization

Authenticated User

→

Cluster Permissions

Controlled by:

EKS Access Entries

RBAC

Lessons Learned

1. EKS cluster creation does not automatically authorize every IAM role.
2. Network access and authorization are separate problems.
3. Successful AWS authentication does not guarantee Kubernetes access.
4. Security Group references are preferred over IP-based rules.
5. EKS Access Entries provide modern cluster access management.
6. Troubleshooting should follow:

Network

→ Authentication

→ Authorization

Final Architecture

